

Coloração de Arestas - Implementação

Paulo Artur Villça

Março de 2026

1 Introdução

O intuito desse artigo é documentar sobre a implementação do algoritmo utilizado para a coloração de arestas

2 Sobre o programa

- Compilação: `make`

Como o programa utiliza de uma dependência para o desenho de coisas na tela precisei utilizar do GNU make para a automação do processo de compilação e linking.

- Uso: `./edger <caminho-do-grafo>`

Quando a compilação for efetuada, um executável `edger` surgirá no diretório raiz do projeto. Existem alguns exemplos que podem ser usados dentro do diretório `samples/`

Exemplo: `./edger samples/basic.txt`

Obs. Caso o grafo estiver malformado, não existir ou não for informado, acontecerá algum comportamento indefinido e possivelmente uma falha de segmentação.

- Dependência: O programa utiliza o raylib como uma biblioteca de gráficos 2d.

Raylib é um wrapper em cima sobre GLFW, OpenGL e Miniaudio, permitindo programação multimídia de fácil acesso em C.

O programa é linkado estáticamente com o Raylib, não sendo necessário baixar algo a mais para seu funcionamento.

3 Funcionamento

O programa segue com a filosofia de evitar ao máximo alocações dinâmicas, pelo menos as explícitas, é esperado que esse programa não funcione para grafos gigantes com mais de 1000 arestas ou algo do tipo, pois C tem um limite máximo para a alocação no stack, não é do meu controle, apenas uma vulnerabilidade/limitação da linguagem.

3.1 Ponto de entrada

O pseudocódigo da função principal segue a seguinte estrutura:

1. Ler o arquivo
2. Converter o arquivo em uma matriz de adjacência (vértice)
3. Converter a matriz de adjacência (vértice) em uma matriz de incidência
4. Converter a matriz de incidência em uma matriz de adjacência (aresta)
5. Colorir as arestas do grafo
6. Desenhar o grafo colorido utilizando Raylib.

3.2 Algoritmo de coloração

O algoritmo escolhido para a coloração é um algoritmo de força bruta recursivo que tem como parâmetro a matriz de adjacência (aresta) e retorna o índice cromático e um vetor contendo as cores de cada aresta.

No código real as coisas podem parecer mais confusas pois eu estou misturando parâmetros de entrada, com parâmetros de saída, com o estado do algoritmo (variáveis globais), novamente, outra limitação da linguagem.

E também antes do algoritmo real é feito um pré-processamento onde é dado um ponto de entrada para o algoritmo, calculado o pior caso e inicialização dos buffers.

3.2.1 Caso recursivo

```
entrada: aresta atual
  para cada cor até o pior caso
    se essa cor não for adjacente a aresta atual
      colorir ela
      se a quantidade de cores necessárias for
        maior que o pior caso
        pular iteração
      caso contrário
        possível melhor coloração = colorir
        próxima aresta com o mesmo algoritmo

        se possível melhor caso for realmente a melhor coloração
          registrála
          tentar próxima cor
retornar melhor coloração
```

3.2.2 Caso propagação

```
se todas as arestas já foram coloridas,
  apenas retornar a melhor coloração feita nesse caminho
```

3.3 Utilidades

3.3.1 Matriz de adjacência (vértice) para matriz de incidência

```
para cada vértice
  para cada outro vértice não visto
    se eles se conectam
      empurrar uma aresta que
        referenciam ambos vértices
```

3.3.2 Matriz de incidência para matriz de adjacência (aresta)

$$I^T \cdot I = O$$

Onde I é a matriz de incidência e O é a matriz de adjacência (aresta)

Após isso é feito um pós-processamento, removendo as auto-adjacências de O e feito um clamping entre $[0, 1]$ sobre os outros elementos da matriz.

3.4 Exemplo do algoritmo

Vamos utilizar o grafo do enunciado da T2:

```
4 5
0 1 1 1
1 0 1 0
1 1 0 1
1 0 1 0
```

Fazendo uma breve análise, esse é um grafo simples, não é regular, nem completo.

De acordo com o Teorema de Vizing é possível assegurar que seja um grafo Classe 1 (índice cromático = grau máximo).

A quantidade máxima de conexões que um vértice faz nesse grafo é 3.

Logo seu índice cromático é 3, ou seja, 3 cores distintas serão utilizadas na coloração desse grafo.

É possível rodar esse exemplo com `./edger samples/basic.txt`.

Os logs estão disponíveis no código fonte da branch `logging git checkout logging`.

```
[COMEÇO] pior caso: 4 // o pré processador assume que o pior caso
sempre é grau máximo + 1
[COLOR] tentando colorir aresta 0 com cor 0
[COLOR] tentando colorir aresta 1 com cor 1
[COLOR] tentando colorir aresta 2 com cor 2
[COLOR] tentando colorir aresta 3 com cor 2
[COLOR] tentando colorir aresta 4 com cor 0
[UPDATE] enquanto colorindo a aresta 4, com a cor 0, uma melhor coloração
foi achada! com índice cromático 3 // nesse ponto a melhor coloração
possível foi achada, agora o algoritmo vai começar a descartar (prune)
qualquer outra coloração e começar propagar a atualização para cima
no stack de chamada.
[COLOR] tentando colorir aresta 4 com cor 3
[PRUNE] enquanto colorindo a aresta 4, uma coloração não ideal com
a cor 3 foi proposta, pulando
[UPDATE] enquanto colorindo a aresta 3, com a cor 2, uma melhor coloração
foi achada! com índice cromático 3 // propagação da aresta 4 para
aresta 3
[COLOR] tentando colorir aresta 3 com cor 3
```

[PRUNE] enquanto colorindo a aresta 3, uma coloração não ideal com a cor 3 foi proposta, pulando
[UPDATE] enquanto colorindo a aresta 2, com a cor 2, uma melhor coloração foi achada! com índice cromático 3 // propagação da aresta 3 para aresta 2
[COLOR] tentando colorir aresta 2 com cor 3
[PRUNE] enquanto colorindo a aresta 2, uma coloração não ideal com a cor 3 foi proposta, pulando
[UPDATE] enquanto colorindo a aresta 1, com a cor 1, uma melhor coloração foi achada! com índice cromático 3 // propagação da aresta 2 para aresta 1
[COLOR] tentando colorir aresta 1 com cor 2
[PRUNE] enquanto colorindo a aresta 1, uma coloração não ideal com a cor 2 foi proposta, pulando
[COLOR] tentando colorir aresta 1 com cor 3
[PRUNE] enquanto colorindo a aresta 1, uma coloração não ideal com a cor 3 foi proposta, pulando
[UPDATE] enquanto colorindo a aresta 0, com a cor 0, uma melhor coloração foi achada! com índice cromático 3 // propagação da aresta 1 para aresta 0
[COLOR] tentando colorir aresta 0 com cor 1
[COLOR] tentando colorir aresta 1 com cor 0
[COLOR] tentando colorir aresta 2 com cor 2
[PRUNE] enquanto colorindo a aresta 2, uma coloração não ideal com a cor 2 foi proposta, pulando
[COLOR] tentando colorir aresta 2 com cor 3
[PRUNE] enquanto colorindo a aresta 2, uma coloração não ideal com a cor 3 foi proposta, pulando
[COLOR] tentando colorir aresta 1 com cor 2
[PRUNE] enquanto colorindo a aresta 1, uma coloração não ideal com a cor 2 foi proposta, pulando
[COLOR] tentando colorir aresta 1 com cor 3
[PRUNE] enquanto colorindo a aresta 1, uma coloração não ideal com a cor 3 foi proposta, pulando
[COLOR] tentando colorir aresta 0 com cor 2
[PRUNE] enquanto colorindo a aresta 0, uma coloração não ideal com a cor 2 foi proposta, pulando
[COLOR] tentando colorir aresta 0 com cor 3

[PRUNE] enquanto colorindo a aresta 0, uma coloração não ideal com a cor 3 foi proposta, pulando

[FIM] índice cromático: 3

Nesse caso o algoritmo finalizou com uma quantidade pequena de passos.

Mas esse não é o caso para muitos grafos, principalmente aqueles Classe 2.

4 Referências

- Raylib
- VIZING'S THEOREM AND EDGE-CHROMATIC GRAPH THEORY